Java – Graphical User Interface (GUI)

Graphical User Interface - GUI
Abstract Window Toolkit (AWT) & Swing

- Agenda
  - O que é AWT e Swing?
  - Programas simples com interface gráfica com usuário
    - Componentes
    - Layouts
    - Composição com painéis
  - Swing X AWT

Graphical User Interface

- Conjunto completo de classes para escrever programas com GUI
  - botões, listas, menus e áreas de texto;
  - containers (como janelas e barras de menus) e
  - componentes de nível mais alto (como diálogos de abertura e salvamento de arquivos)
  - imagens, eventos, fontes e cores
  - gerenciadores de layout, que controlam o tamanho e a posição dos componentes

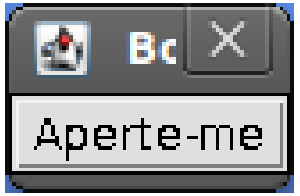
Abstract Window Toolkit (AWT) Características Principais

- Utiliza os controles padrões do ambiente de execução
  - Os controles são desenhados pelo SO
- Conjunto básico de controles
  - Controles existentes nos ambientes gráficos suportados
- O programa gerado apresenta os aspectos visuais de costume oferecidos pelo SO

Componentes AWT

- Button

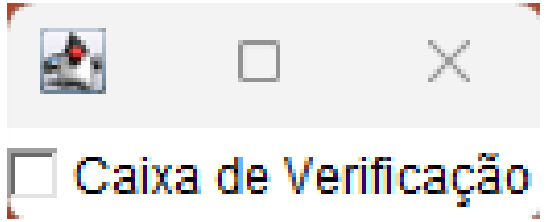
```
import java.awt.*;

public class Botao extends Frame {
    Button b = new Button("Aperte-me");

    Botao() {
        super("Botao");
        add(b);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new Botao();
    }
}
```



Componentes AWT

- CheckBox

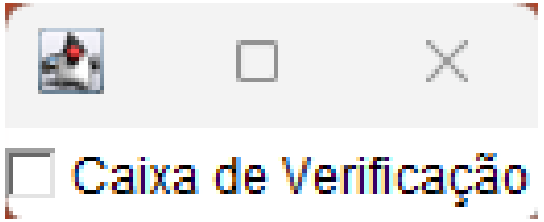
```
import java.awt.*;
class CaixaVerif extends Frame {
    Checkbox cb = new Checkbox("Caixa de Verificação");

    CaixaVerif( ) {
        super("Checkbox");
        add(cb);
        pack( );
        setVisible(true);
    }

    static public void main(String[] args) {
        new CaixaVerif( );
    }
}
```



Componentes AWT

- CheckBox

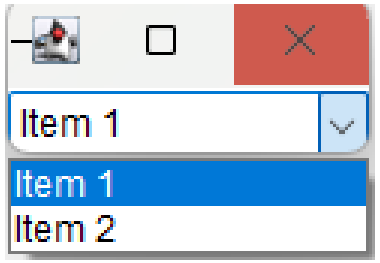
```
import java.awt.*;

class CaixaVerif1 {
    static public void main(String[] args) {
        Checkbox cb = new Checkbox("Caixa de Verificação");
        Frame f = new Frame("CheckBox");
        f.add(cb);
        f.pack( );
        f.setVisible(true);
    }
}
```



Componentes AWT

- Choice

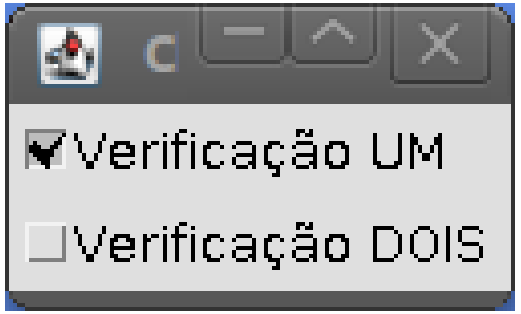
```
import java.awt.*;

class Escolha extends Frame {
    Escolha() {
        super("Choice");
        Choice choice = new Choice();
        choice.addItem("Item 1");
        choice.addItem("Item 2");
        add(choice);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new Escolha();
    }
}
```

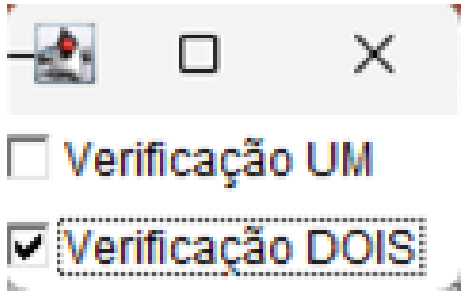


Componentes AWT

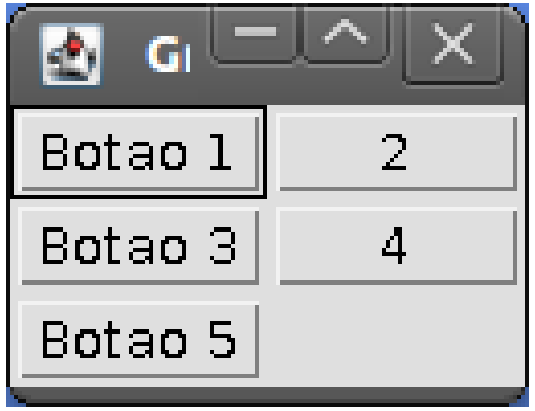
- CheckBox + GridLayout

```
import java.awt.*;

class CaixaVerif2 extends Frame {
    Checkbox cb1 = new Checkbox("Verificação UM");
    Checkbox cb2 = new Checkbox("Verificação DOIS");

    CaixaVerif2() {
        super("Checkbox");
        setLayout(new GridLayout(2, 1));
        add(cb1);
        add(cb2);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new CaixaVerif2();
    }
}
```



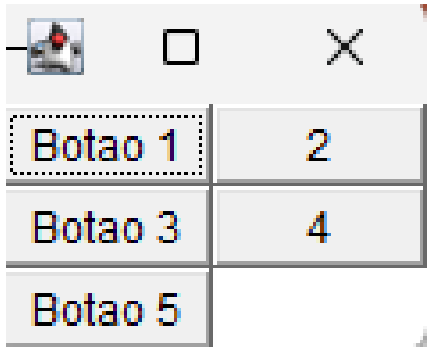


Componentes AWT



- GridLayout

```
import java.awt.*;

class LayoutGrade extends Frame {
    LayoutGrade() {
        super("GridLayout");
        setLayout(new GridLayout(3, 2));
        add(new Button("Botao 1"));        add(new Button("2"));
        add(new Button("Botao 3"));        add(new Button("4"));
        add(new Button("Botao 5"));
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new LayoutGrade();
    }
}
```



Swing
Características Principais

- Não utiliza os controles padrões do ambiente de execução
  - Os controles são desenhados pelo Java
- Conjunto maior de controles
  - árvores, botões com imagens, tabelas, sliders, ...
- Maior versatilidade na manipulação visual dos controles
  - Admite alteração visual dos controles
  - *Look and feel*

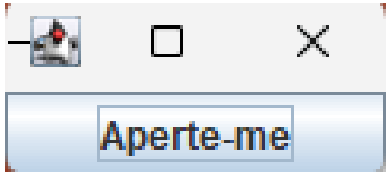
Swing Diferenças para o AWT

- Componentes com nomes iniciados por J
- Todo componente AWT tem um componente Swing compatível
- Não se adiciona os componentes diretamente ao JFrame, usa-se a instância **`contentpane`** já existente no JFrame
- Não misture AWT com Swing

Componentes Swing

- JButton

```
import javax.swing.*;

public class SBotao extends JFrame {
    JButton b = new JButton("Aperte-me");

    SBotao() {
        super("Botao Swing");
        add(b);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new SBotao();
    }
}
```



Componentes Swing

- JCheckBox

```
import javax.swing.*;

public class SCaixaVerif extends JFrame {
    JCheckBox cb = new JCheckBox("Caixa de Verificação");

    SCaixaVerif() {
        super("Checkbox");
        add(cb);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new SCaixaVerif();
    }
}
```

Componentes Swing

- JCheckBox

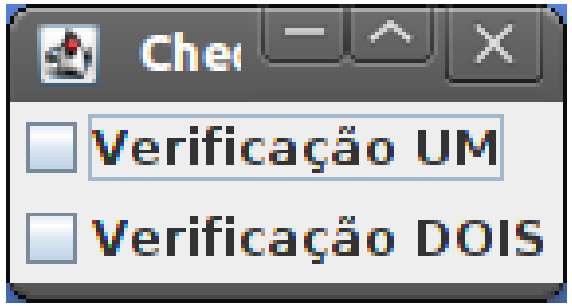
```
import javax.swing.*;

public class SCaixaVerif1 {
    static public void main(String[] args) {
        JCheckBox cb = new JCheckBox("Caixa de Verificação");
        JFrame f = new JFrame("CheckBox");
        f.add(cb);
        f.pack();
        f.setVisible(true);
    }
}
```

Componentes Swing

- JComboBox

```
import javax.swing.*;

class SEscolha extends JFrame {
    SEscolha() {
        super("Choice");
        JComboBox choice = new JComboBox();

        choice.addItem("Item 1");
        choice.addItem("Item 2");
        add(choice);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new SEscolha();
    }
}
```

Layouts



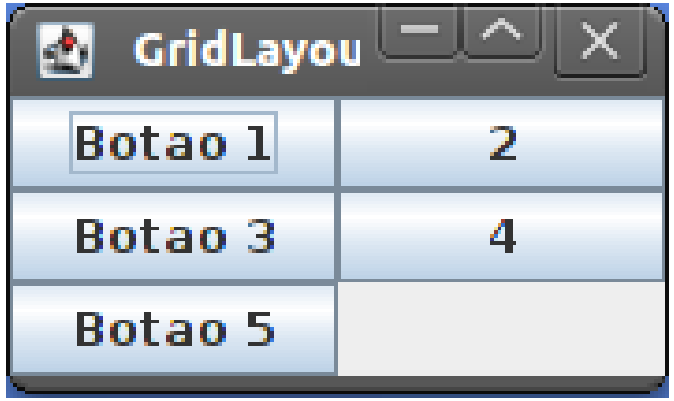

```
import java.awt.*;
import javax.swing.*;

class SCaixaVerif2 extends JFrame {
    JCheckBox cb1 = new JCheckBox("Verificação UM");
    JCheckBox cb2 = new JCheckBox("Verificação DOIS");

    SCaixaVerif2() {
        super("Checkbox");
        setLayout(new GridLayout(2, 1));
        add(cb1);
        add(cb2);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new SCaixaVerif2();
    }
}
```

Componentes Swing

- JRadioButton

```
import java.awt.*;
import javax.swing.*;

class SBotaoRadio extends JFrame {
    JRadioButton rb1 = new JRadioButton("Radio UM");
    JRadioButton rb2 = new JRadioButton("Radio DOIS");

    SBotaoRadio() {
        super("Botão de Rádio");
        setLayout(new GridLayout(2, 1));
        ButtonGroup grupo = new ButtonGroup();
        grupo.add(rb1);
        grupo.add(rb2);
        add(rb1);
        add(rb2);
        pack();
        setVisible(true);
    }

    static public void main(String[] args) {
        new SBotaoRadio();
    }
}
```

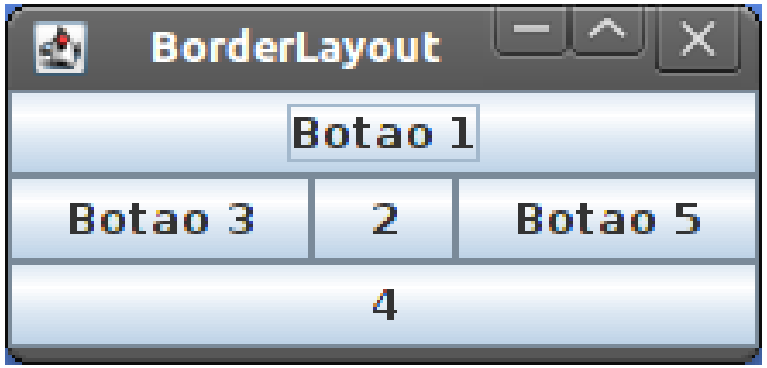
GridLayout



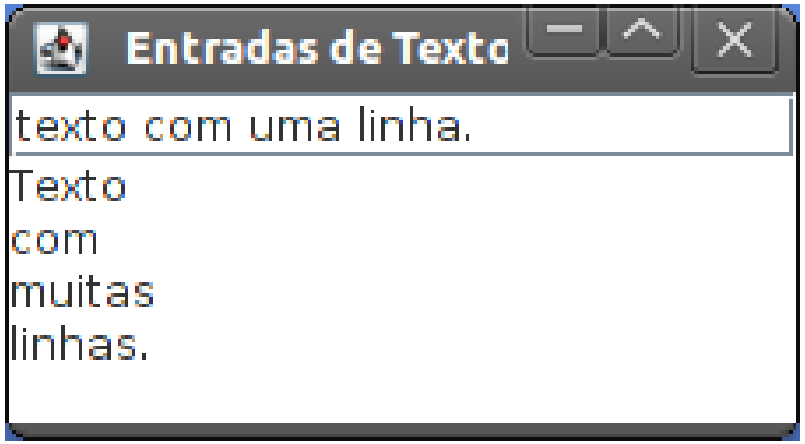

```
import java.awt.*;
import javax.swing.*;

class SLayoutGrade extends JFrame {
    SLayoutGrade( ) {
        super("GridLayout");
        setLayout(new GridLayout(3, 2));
        add(new JButton("Botao 1"));          add(new JButton("2"));
        add(new JButton("Botao 3"));          add(new JButton("4"));
        add(new JButton("Botao 5"));
        pack( );
        setVisible(true);
    }

    static public void main(String[] args) {
        new SLayoutGrade( );
    }
}
```

BorderLayout (layout padrão do JFrame)

```
import java.awt.*;
import javax.swing.*;
class SLayoutBorda extends JFrame {

    SLayoutBorda( ) {
        super("BorderLayout");
        setLayout(new BorderLayout( ));
        add(new JButton("Botao 1"), BorderLayout.NORTH);
        add(new JButton("2"), BorderLayout.CENTER);
        add(new JButton("Botao 3"), BorderLayout.WEST);
        add(new JButton("4"), BorderLayout.SOUTH);
        add(new JButton("Botao 5"), BorderLayout.EAST);
        pack(); setVisible(true);
    }

    static public void main(String[] args) {
        new SLayoutBorda( );
    }
}
```

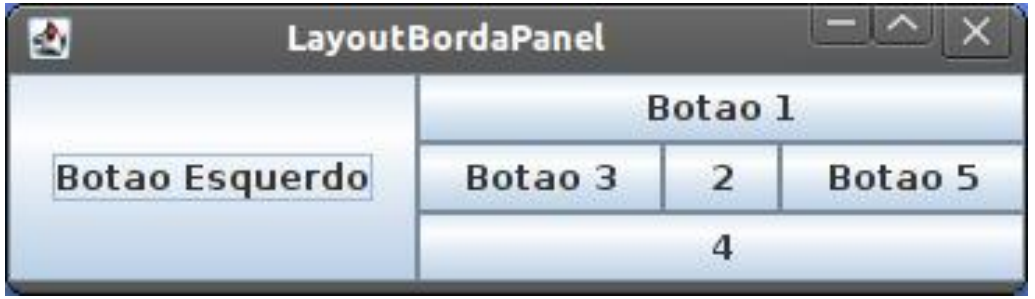


BorderLayout
TextAreas e TextFields




```
import java.awt.*;
import javax.swing.*;

class STexto extends JFrame {
    JTextField textField;
    JTextArea textArea;

    STexto( ) {
        super("Entradas de Texto");
        add(textField = new JTextField(20), BorderLayout.NORTH);
        add(textArea = new JTextArea(5, 20), BorderLayout.CENTER);
        pack( );
        setVisible(true);
    }

    static public void main(String[] args) {
        new STexto( );
    }
}
```

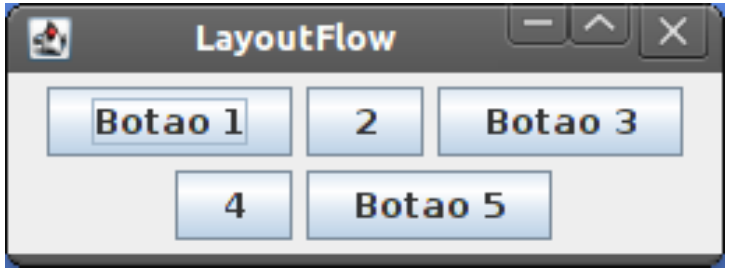
Layouts Complexos com Panels

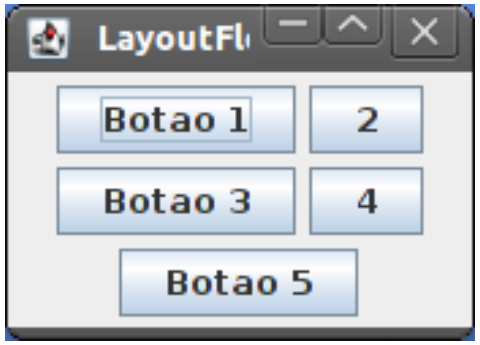



```
import java.awt.*;
import javax.swing.*;
class SLayoutBordaPanel extends JFrame {

    SLayoutBordaPanel( ) {
        super("LayoutBordaPanel");
        JPanel p = new JPanel(new BorderLayout( ));
        p.add(new JButton("Botao 1"), BorderLayout.NORTH);
        p.add(new JButton("2"), BorderLayout.CENTER);
        p.add(new JButton("Botao 3"), BorderLayout.WEST);
        p.add(new JButton("4"), BorderLayout.SOUTH);
        p.add(new JButton("Botao 5"), BorderLayout.EAST);

        add(new JButton("Botao Esquerdo"), BorderLayout.WEST);
        add(p, BorderLayout.EAST);

        pack();  setVisible(true);
    }
    static public void main(String[] args) {
        new SLayoutBordaPanel( );
    }
}
```

FlowLayout (layout padrão do Panel)

```java
import java.awt.*;
import javax.swing.*;

class SLayoutFluxo extends JFrame {
    SLayoutFluxo( ) {
        super("LayoutFlow");
        setLayout(new FlowLayout( ));
        add(new JButton("Botao 1"));
        add(new JButton("2"));
        add(new JButton("Botao 3"));
        add(new JButton("4"));
        add(new JButton("Botao 5"));
        pack();  setVisible(true);
    }

    static public void main(String[] args) {
        new SLayoutFluxo( );
    }
}
```





Swing X AWT

- AWT
  - Mais velocidade e menos memória usada
  - Os componentes visuais são os disponíveis no ambiente onde a máquina virtual está executando
    - Isso cria problemas em plataformas onde nem todos os componentes estão disponíveis

Swing X AWT

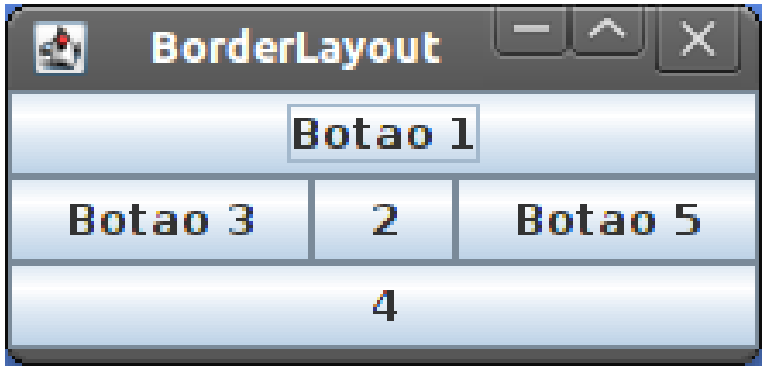
- Swing
  - Desenvolvido em Java puro apresenta poucas alterações na migração entre ambientes
    - Como os componentes são desenhados pelo Java, as limitações de aparência são pequenas
    - Necessita de mais processamento e mais memória
  - Maior variedade de tipos de componentes
  - Look and Feel
    - Aparências podem ser escolhidas e podem ser mantidas para aplicações em qualquer ambiente
    - Pode imitar as aparências dos controles do ambiente

JFrame

- JFrame contém um único componente do tipo JRootPane (chamado contentpane).
- Para adicionar um componente a um JFrame podemos fazer:
  - **`jframe.getContentPane().add(child);`**

  ou
  - JFrame contém métodos **`add(...)`** que adicionam componentes diretamente ao contentpane.

contentpane




```
import java.awt.*;
import javax.swing.*;
class SLayoutBorda1 extends JFrame {

    SLayoutBorda1( ) {
        super("BorderLayout");
        Panel p = new Panel(new BorderLayout( ));
        p.add(new JButton("Botao 1"), BorderLayout.NORTH);
        p.add(new JButton("2"), BorderLayout.CENTER);
        p.add(new JButton("Botao 3"), BorderLayout.WEST);
        p.add(new JButton("4"), BorderLayout.SOUTH);
        p.add(new JButton("Botao 5"), BorderLayout.EAST);
        setContentPane(p);
        pack(); setVisible(true);
    }

    static public void main(String[] args) {
        new SLayoutBorda1( );
    }
}
```

Fechamento de JFrame

- O que acontece quando se tenta fechar a janela?
  - AWT – Nada!
    - É necessário substituir (override) o handleEvent() de Frame.
  - Swing – a janela é escondida!
    - usa-se setDefaultCloseOperation(operação)
      - Onde operação:
        - WindowConstants.DISPOSE_ON_CLOSE ou
        - WindowConstants.EXIT_ON_CLOSE
    - trata-se os eventos correspondentes ao fechamento da janela com Listeners

Fechamento de JFrame

```
import javax.swing.*;

public class SClose extends JFrame {
    JButton b = new JButton("Aperte-me");

    SClose( ) {
        super("Botao Swing");
        add(b);
        pack( );
        setVisible(true);
        setDefaultCloseOperation(WindowConstants.DISPOSE_ON_CLOSE);
    }

    static public void main(String[] args) {
        new SClose( );
    }
}
```